Glia

Functional Requirements Specification

A library for living semantic cache management in RAG architectures

1. Overview

Glia is a Python library that wraps around an existing semantic caching workflow to add three capabilities: dependency-tagged cache storage, targeted cache invalidation via a background watcher, and developer-controlled embedding injection. The library is designed to sit alongside an existing notebook without requiring changes to how the developer parses documents, generates FAQs, or interacts with the LLM. The developer adds a small initialization block and a single metadata argument to the existing storage call. Everything else in the workflow continues unchanged.

Attribute	Value
Primary Language	Python 3.9+
Cache Backend	Redis Stack (RediSearch module required)
Embedding Provider	Injected by the developer — not managed by the library
Integration Style	Additive wrapper — does not modify existing workflow code
Vectorizer Baseline	textembedding-gecko@003 (768 dimensions, configurable)

2. Module Structure

The library is organized into three self-contained modules. Each module can be used independently, but they are designed to work together.

2.1 GliaManager

Handles storing new cache entries with tagged metadata and looking up cached responses. This module is initialized once at the top of the notebook and passed the developer's existing vectorizer.

FR-2.1.1 Embedding Provider Injection

- The library MUST accept an embedding provider object at initialization time via a vectorizer= constructor parameter.

- The library MUST NOT instantiate or import any embedding provider internally.
- The developer is responsible for constructing the provider before passing it in.
- The injected provider MUST expose an embed() method for single-query lookup and an embed_many() method for batch ingestion.

FR-2.1.2 Custom Schema with source_id Tag

- The library MUST define a default schema that includes: prompt (text), response (string), source_id (tag), and prompt_vector (vector).
- Vector dimensions MUST be configurable to match the injected provider (default: 768).
- The library MUST support an optional custom_schema parameter to extend the default schema with additional tag or numeric fields.
- Entries MUST be serialized as JSON documents stored under a configurable key prefix, compatible with RediSearch JSON indexing.

FR-2.1.3 store() Method

- The library MUST provide a store(prompt, response, source_id, metadata=None) method.
- On each call, store() MUST embed the prompt using the injected provider, generate a cache key, write the JSON payload to Redis, and ensure the entry is indexed under the active RediSearch index.
- The source_id value MUST be stored as a TAG field so it can be queried and targeted during invalidation.

FR-2.1.4 check() Method

- The library MUST provide a check(prompt, filter=None) method for cache lookup.
- The method MUST embed the prompt and perform a vector similarity search against prompt_vector.
- The distance_threshold MUST be configurable at initialization (default: 0.2).
- The optional filter parameter MUST allow pre-filtering results by any TAG field before similarity scoring is applied.
- The method MUST return the cached response string on a hit, or None on a miss.

2.2 CacheInvalidator

Handles targeted removal of stale cache entries. This module is invoked by the CacheWatcher when a source document changes, but can also be called directly by the developer.

FR-2.2.1 delete_by_tag() Method

- The library MUST provide a delete_by_tag(source_id) method on the CacheInvalidator class.
- The method MUST query the RediSearch index for all entries where the source_id TAG field matches the provided value, and delete all matched entries in a single batched operation.

- The method MUST return the count of deleted entries and be idempotent: if no entries are found, it MUST return 0 without raising an error.

FR-2.2.2 Atomic Cleanup

- When entries are deleted, both the JSON data and the corresponding RediSearch index references MUST be removed together.
- The library MUST NOT leave orphaned index entries that could return ghost results in future searches.

2.3 CacheWatcher

A background service that monitors data sources and triggers invalidation automatically when changes are detected.

FR-2.3.1 File System Monitoring

- The watcher MUST be able to monitor one or more local file system directories for changes.
- The watcher MUST detect file modification and deletion events.
- On detection, the watcher MUST derive the source_id for the changed file and pass it to delete_by_tag().

FR-2.3.2 Background Execution

- The watcher MUST run in a background thread so it does not block the main workflow.
- The watcher MUST expose a start() method to begin monitoring and a stop() method to cleanly terminate.
- The watcher MUST be safe to run inside a Jupyter Notebook environment (nest_asyncio compatible).

FR-2.3.3 Event Callback

- The watcher MUST support an optional on_invalidation callback parameter.
- If provided, the callback MUST be invoked after each successful delete_by_tag() call, receiving the source_id and deleted entry count as arguments.

---

3. Integration Contract with the Workflow

The library is designed as an additive wrapper. The developer's existing workflow — document parsing, node creation, FAQ generation, and LLM interaction — continues unchanged. Glia is integrated by adding a single initialization block and tagging the existing storage call with a source_id. No other lines in the workflow need to be modified.

3.1 What Does Not Change

The following parts of the existing workflow MUST require zero modification after the library is introduced:

- The document loading and parsing pipeline (e.g. LlamaParse, MarkdownNodeParser, node iteration).
- The LLM setup and FAQ generation chain (e.g. ChatVertexAI, PromptTemplate, LangChain chain invocation).
- The Redis connection setup (REDIS_HOST, REDIS_PORT, connection string).
- The vectorizer initialization (e.g. VertexAITextVectorizer with project_id and region). The developer passes this existing instance into GliaManager — it is not re-created.
- The overall execution flow: parse → generate FAQs → store → check on query. The sequence does not change.

3.2 What the Developer Adds

After installing the library, the developer makes the following additions to the existing notebook. These are the only changes required.

Addition 1 — Library initialization (one new block)

After the existing vectorizer is initialized, the developer adds a single new cell to construct the GliaManager. The vectorizer instance the developer already created is passed directly into the constructor. No new embedding configuration is needed.

```
from glia import GliaManager, CacheInvalidator, CacheWatcher

cache_manager = GliaManager (
    vectorizer=vectorizer,          # the existing
VertexAITextVectorizer instance
    redis_url=REDIS_URL,            # the existing Redis connection
string
    distance_threshold=0.2          # same threshold used previously
)
```

Addition 2 — source_id tag on the existing store call (one new argument)

Inside the existing FAQ storage loop, the developer adds source_id=node.id_ to the store() call. The loop structure, chain invocation, and iteration over results do not change.

```
# Before (existing code, unchanged):
for i, node in enumerate(nodes):
    results = faq_generator_chain.invoke({"doc": node.text})
    if results and results.get("pairs"):
```

```
        for entry in results["pairs"]:
            cache.store(entry["prompt"], entry["response"])  # original
call

# After (only the store call gains one argument):
            cache_manager.store(
                entry["prompt"],
                entry["response"],
                source_id=node.id_    # the only addition
            )
```

Addition 3 — Watcher startup (one new block, optional)

If the developer wants the background watcher, they add a single cell to start it. This cell is independent of all other workflow code and can be added at the end of the notebook or run as a standalone script.

```
invalidator = CacheInvalidator(cache_manager)
watcher = CacheWatcher(invalidator, watch_dirs=["data/"])
watcher.start()
```

3.3 Replacement of cache.check()

The developer replaces the existing cache.check() call with cache_manager.check(). The method signature and return behavior are identical — a matching response string is returned on a hit, None on a miss — so no surrounding logic changes.

```
# Before:
result = cache.check(prompt=user_query)

# After:
result = cache_manager.check(user_query)
```

---

4. Initialization Interface

The following parameters MUST be accepted by GliaManager at initialization.

Parameter	Description
vectorizer (required)	The developer-instantiated embedding provider. Must expose embed() and embed_many() methods.

redis_url (required)	Connection string for the Redis Stack instance (e.g. redis://localhost:6379).
index_name (optional)	Name of the RediSearch index. Defaults to llmcache.
distance_threshold (optional)	Float controlling semantic match precision. Defaults to 0.2.
vector_dims (optional)	Embedding dimensions. Defaults to 768 for textembedding-gecko@003.
custom_schema (optional)	Additional field definitions (tag or numeric) to extend the default schema.
ttl_seconds (optional)	If set, cache entries expire after this many seconds, preventing orphaned entries if the watcher fails.

---

5. Constraints and Boundaries

The following are intentionally out of scope for the library.

- The library MUST NOT manage or import any LLM. LLM interaction remains entirely in the developer's code.
- The library MUST NOT perform document loading or parsing. Node creation via LlamaIndex remains external.
- The library MUST NOT manage FAQ generation chains. The LangChain prompt chain remains external.
- The library MUST require Redis Stack with the RediSearch module. Standard Redis without Search is not supported.
- File system monitoring is limited to locally accessible directories. Remote object storage events (e.g. S3) are out of scope for v1.
- The library MUST NOT auto-select or swap the embedding model. The injected vectorizer is the sole embedding source for the lifetime of the cache instance.

---

6. Observability

The library MUST emit the following events to support debugging and auditing without requiring a third-party monitoring tool.

Event	Payload
cache_hit	source_id of the matched entry, similarity score
cache_miss	Queried prompt (truncated to 200 characters)

invalidation_complete	source_id, count of deleted entries
watcher_event	File path, event type (modified / deleted)

All events MUST follow a consistent dictionary structure passable to the developer's on_invalidation callback or a standard Python logging handler. The library MUST NOT impose a specific logging framework.

---

7. Zero-Config Fallback

The library MUST support a minimal initialization path where only vectorizer= and redis_url= are supplied and all optional features default to sensible values. In this mode the library behaves as a drop-in for the existing semantic cache, with source_id tagging and targeted invalidation available but not required on first use.

```
glia/
│
├── __init__.py
├── manager.py
├── invalidator.py
├── schema.py
├── watcher.py
├── runners.py
├── events.py
│
├── adapters/
│   ├── __init__.py
│   ├── base.py
│   ├── polling.py
│   ├── cdc.py
│   ├── vector.py
│   ├── graph.py
│   └── relational.py
│
└── exceptions.py
```

manager.py — GliaManager The entry point most developers interact with. Owns the Redis connection, the RediSearch index lifecycle, and the store() / check() methods.

schema.py — SchemaBuilder Kept separate because schema construction is pure configuration logic with no I/O. GliaManager instantiates it internally, but isolating it makes it independently testable and easy to extend with new field types later.

invalidator.py — CacheInvalidator Single responsibility: take a source_id, find matching cache keys via RediSearch, delete them atomically. It receives a GliaManager instance at init so it shares the same Redis connection — no second connection opened.

watcher.py — CacheWatcher Orchestration only. Inspects each adapter's mode at start(), creates the right runner for it, and holds references to all running threads. No polling or stream logic lives here.

runners.py — PollingRunner, CDCRunner The two execution engines. Each owns one adapter and one background thread. PollingRunner runs the timed loop; CDCRunner runs the blocking stream listener. Both call back into CacheWatcher._dispatch() when a source_id is surfaced.

events.py — EventEmitter, WatcherEvent Fully decoupled from everything else. Any component that needs to fire an event gets an EventEmitter injected — it never imports from manager.py or watcher.py.

adapters/base.py — DatabaseAdapter The abstract base class. Defines mode, source_id_field, connect(), disconnect(), and map_to_source_id(). Every concrete adapter inherits from this.

adapters/polling.py — PollingAdapter Extends DatabaseAdapter. Adds the polling contract: poll(), get_cursor(), advance_cursor(), poll_interval, last_cursor. This is what PollingRunner types against — it never imports a concrete adapter directly.

adapters/cdc.py — CDCAdapter Extends DatabaseAdapter. Adds the CDC contract: listen() (a generator), stop(), reconnect_retries, reconnect_delay. This is what CDCRunner types against.

adapters/vector.py — VectorDBAdapter Inherits from both PollingAdapter and CDCAdapter. Implements both contracts so the developer can instantiate it with either mode="polling" or mode="cdc". Same pattern for graph and relational.

adapters/graph.py — GraphDBAdapter Same dual-inheritance pattern. CDC implementation uses Neo4j's transaction log or CDC connector; polling uses a Cypher query against last_modified.

adapters/relational.py — RelationalDBAdapter Same dual-inheritance pattern. CDC uses logical replication / binlog streaming; polling queries updated_at or a change-log table.

exceptions.py — custom exceptions (MissingModeError, AdapterConnectionError, SchemaValidationError, InvalidationError) Centralised so any module can from glia.exceptions import ... without circular imports. The MissingModeError is specifically raised in DatabaseAdapter.__init__() when mode= is not supplied.

__init__.py — public surface Re-exports only what the developer needs to import directly:

python

```
from glia import GliaManager, CacheInvalidator, CacheWatcher

from glia.adapters import VectorDBAdapter, GraphDBAdapter, RelationalDBAdapter
```

Everything else (runners.py, events.py, schema.py) is internal — used by the library but not part of the public API a developer is expected to touch.

V

visualize show_widget

Click any file in the tree to expand its detail. A few structural decisions worth highlighting:

runners.py imports PollingAdapter and CDCAdapter from the abstract files — never the concrete adapters. This means adding a new database type in the future (e.g. DocumentDBAdapter) requires zero changes to the runner files.

events.py has zero imports from the rest of glia/. It's the one file that can be tested in complete isolation and reused elsewhere if needed.

exceptions.py is at the top level rather than inside adapters/ because manager.py and watcher.py also raise some of these — centralising them prevents the circular import that would happen if manager.py imported from adapters/.

Here's the implementation order, structured in layers — each layer only depends on what's already been built.

Layer 1 — Foundation (no dependencies)

Start here because everything else imports from these files. Nothing in this layer imports from any other glia module.

1. exceptions.py — define all custom exceptions first. Every other file will import from here.
2. events.py — WatcherEvent dataclass, then EventEmitter. No glia imports at all.
3. schema.py — SchemaBuilder. Only depends on redisvl, no glia imports.

Layer 2 — Adapter contracts

The abstract base classes. Implement these before any concrete adapters or the watcher, because runners and concrete adapters type against these interfaces.

4. adapters/base.py — DatabaseAdapter ABC. Imports only from exceptions.py.
5. adapters/polling.py — PollingAdapter ABC. Extends DatabaseAdapter.
6. adapters/cdc.py — CDCAdapter ABC. Extends DatabaseAdapter.

Layer 3 — Cache core

Now that the schema is ready, build the cache manager and invalidator.

7. manager.py — GliaManager. Depends on SchemaBuilder and your Redis/redisvl connection.
8. invalidator.py — CacheInvalidator. Depends on GliaManager. Keep it simple — just delete_by_tag() wired up and tested before moving on.

Layer 4 — Concrete adapters

Each one is self-contained. Implement and test them one at a time — order doesn't matter within this layer.

9. adapters/vector.py — VectorDBAdapter. Implement polling mode first, then CDC.
10. adapters/graph.py — GraphDBAdapter. Same — polling first, CDC second.
11. adapters/relational.py — RelationalDBAdapter. Same pattern.

The reason to do polling before CDC in each adapter: polling is easier to verify locally (just run a query and inspect the result), while CDC requires a live stream connection and reconnect logic that's harder to test in isolation.

Layer 5 — Watcher engine

Only build this once at least one concrete adapter is working end-to-end.

12. runners.py — PollingRunner first (easier to test — just verify the loop fires and calls _dispatch), then CDCRunner.

13. watcher.py — CacheWatcher. By this point all its dependencies exist. Wire start() to inspect adapter.mode and spawn the right runner.

Layer 6 — Public surface

Last step once everything is working.

14. adapters/__init__.py — re-export VectorDBAdapter, GraphDBAdapter, RelationalDBAdapter.

15. glia/__init__.py — re-export GliaManager, CacheInvalidator, CacheWatcher. Write this last so you only expose what's actually tested and stable.One practical tip for Layer 4: before writing vector.py, manually verify that CacheInvalidator.delete_by_tag() actually removes entries from Redis with a quick script. If that's broken, every adapter test you write on top of it will give you misleading results. Lock down Layer 3 completely before touching Layer 4.